# UCK 439E Term Project
# Optimal B737–800 Flight Planning under a Wind Field

Yunus Emre Kara
Istanbul Technical University

June 2026

**Abstract**

We solve the cruise-phase trajectory-optimisation problem for a Boeing 737–800 under a polynomial wind field with two complementary numerical optimal-control techniques: (i) direct transcription with Hermite–Simpson collocation and IPOPT, and (ii) a reinforcement-learning pipeline that combines multi-expert behavioural cloning of the direct solution with DAgger-style on-policy expert correction in a custom Gymnasium environment. Three flight scenarios from the project specification are solved by both methods, eight standard plots are produced per flight, and a single overlay figure per metric compares the two methods. The direct method serves as the optimal-control benchmark; the RL pipeline produces a state-feedback policy that generalises across an entire ball of initial conditions around each flight plan.

An earlier reduced-state dynamic-programming module (constant cruise altitude / airspeed / throttle on a 2-D ENU grid) was implemented and later removed: collapsing the 6-D state to 2-D made DP numerically fast but produced trajectories that ignore the altitude and energy management that dominate the PDF cost, so DP was not a like-for-like comparison against direct collocation. See Sec. 4 for the analysis behind that decision.

## 1 Introduction

This report documents the term project for UCK 439E (Introduction to Optimal Control). The task statement is a B737–800 flight-planning problem with a known wind field $W(\lambda, \varphi)$ given by a ninth-order polynomial in (longitude, latitude). The aircraft performance model is BADA-style with ISA-tropospheric atmosphere. Three flight scenarios are prescribed (Tables 1 and 2); each scenario fixes the start state $(\lambda_0, \varphi_0, h_0, V_0, \psi_0, m_0)$ and the destination $(\lambda_f, \varphi_f, h_f)$.

Table 1: Flight scenarios from the project specification. Heading $\psi_0$ follows the math-from-east convention used throughout this work (see §2.1).

| flight | initial state | | | | | | destination | | |
|---|---|---|---|---|---|---|---|---|---|
| | $\lambda_0$ [deg] | $\varphi_0$ [deg] | $h_0$ [m] | $V_0$ [m/s] | $\psi_0$ [deg] | $m_0$ [kg] | $\lambda_f$ [deg] | $\varphi_f$ [deg] | $h_f$ [m] |
| 1 | −4.0 | 40.0 | 8000 | 200 | 0 | 71000 | 30.0 | 52.0 | 9000 |
| 2 | 30.0 | 50.0 | 10000 | 220 | 225 | 56000 | 20.0 | 45.0 | 9000 |
| 3 | 15.0 | 40.0 | 9000 | 210 | 0 | 66000 | 15.0 | 53.0 | 7000 |

**Methods chosen.** The project requires (a) one direct or indirect optimal-control method and (b) either dynamic programming *or* reinforcement learning. We implement (a) as Hermite–Simpson direct collocation solved by IPOPT (Sec. 3), and (b) as a behavioural-cloning + DAgger reinforcement-learning pipeline that warm-starts from the direct expert (Sec. 5). DP is discussed briefly in Sec. 4 as a method that was tried and deliberately dropped from the final comparison.

Table 2: Polynomial wind-field coefficients (Term Project PDF Table 2). Coefficient order is $[c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7, c_8]$ in the expansion $W = c_0 + c_1\lambda + c_2\varphi + c_3\lambda\varphi + c_4\lambda^2 + c_5\varphi^2 + c_6\lambda^2\varphi + c_7\lambda\varphi^2 + c_8\lambda^2\varphi^2$ with output in m/s.

| | $c_0$ | $c_1$ | $c_2$ | $c_3$ | $c_4$ | $c_5$ | $c_6$ | $c_7$ | $c_8$ |
|---|---|---|---|---|---|---|---|---|---|
| $W_x$ | $-21.151$ | 10.004 | 1.108 | $-0.524$ | $-0.130$ | $-0.006$ | 0.0073 | 0.0066 | $-1.0\times10^{-4}$ |
| $W_y$ | $-65.304$ | 17.615 | 1.086 | $-0.700$ | $-0.551$ | $-0.003$ | 0.0241 | 0.0064 | $-2.3\times10^{-4}$ |

# 2 Problem Formulation

## 2.1 State, control, and dynamics

The state vector is $\mathbf{x} = (x_e,\, y_n,\, h,\, V,\, \psi,\, m)^\top$ in flat-Earth East–North–Up (ENU) coordinates anchored at the per-flight reference origin $(\lambda_0, \varphi_0)$. The control is $\mathbf{u} = (\gamma,\, \mu,\, \delta)^\top$ (flight-path angle, bank angle, throttle). The PDF dynamics are

$$\begin{aligned}
\dot{x}_e &= V\cos\psi\cos\gamma + W_x(\lambda,\varphi),\\
\dot{y}_n &= V\sin\psi\cos\gamma + W_y(\lambda,\varphi),\\
\dot{h} &= V\sin\gamma,\\
\dot{V} &= \frac{T-D}{m} - g_0\sin\gamma,\\
\dot{\psi} &= \frac{C_L\,S\,\rho V}{2m}\,\frac{\sin\mu}{\cos\gamma},\\
\dot{m} &= -f(V,h,\delta).
\end{aligned} \tag{1}$$

**Heading-angle convention.** Because the kinematic equations use $\dot{x} = V\cos\psi$ and $\dot{y} = V\sin\psi$ with $(x, y) = (\text{east}, \text{north})$, $\psi$ is measured *counter-clockwise from east* (mathematics convention), not clockwise from north (aviation convention). Hence $\psi = 0 \Rightarrow$ heading east; $\psi = \pi/2 \Rightarrow$ heading north. Table 3 entries such as "$\psi_0 = 0\,\text{deg}$" therefore mean an initial heading due east. This choice is consistent across all three methods.

## 2.2 Aerodynamic and propulsion sub-models

Atmosphere $\rho(h) = \rho_0(1-2.2257\times10^{-5}h)^{4.2586}$. Lift coefficient is closed-form from the coordinated-turn balance $L\cos\mu = mg$, giving $C_L = 2mg/(\rho S V^2\cos\mu)$, with polar drag $C_D = C_{D0} + kC_L^2$ and $D = \frac{1}{2}\rho V^2 S C_D$. Maximum thrust uses the BADA polynomial in $h_{\text{ft}}$, $T_{\max}(h) = C_{T,\text{cr}}\,C_{T,c1}(1 - h_{\text{ft}}/C_{T,c2} + C_{T,c3}h_{\text{ft}}^2)$, and the available thrust is $T = \delta\,T_{\max}(h)$. Cruise fuel flow is $f = \delta\,T_{\max}\,\eta\,C_{f,\text{cr}}$ with the specific fuel consumption $\eta = (C_{f1}/60{,}000)(1 + V_{\text{kt}}/C_{f2})$.

## 2.3 Wind field

The wind field is given by Table 2 of the project as a 9-term bivariate polynomial in $(\lambda, \varphi)$ in degrees,

$$W_{\{x,y\}}(\lambda,\varphi) = c_0 + c_1\lambda + c_2\varphi + c_3\lambda\varphi + c_4\lambda^2 + c_5\varphi^2 + c_6\lambda^2\varphi + c_7\lambda\varphi^2 + c_8\lambda^2\varphi^2.$$

Because we integrate dynamics in ENU, we convert $(x_e, y_n) \to (\lambda, \varphi)$ inside the right-hand side using the flat-Earth approximation $\lambda = \lambda_0 + (x_e/(R\cos\varphi_0))\cdot 180/\pi$, $\varphi = \varphi_0 + (y_n/R)\cdot 180/\pi$. Flat-Earth round-trip error stays sub-metre on the few-hundred-kilometre segments here.

### 2.4 Cost functional

The PDF cost is

$$J = \big(x(t_f) - x_d\big)^2 + \big(y(t_f) - y_d\big)^2 + \big(h(t_f) - h_d\big)^2 + \int_0^{t_f} (0.05 + f)\, dt. \tag{2}$$

The Mayer terminal squared-distance term enforces arrival; the running term penalises both fuel and elapsed time (the constant 0.05 has units [kg/s] and biases against needlessly long trajectories).

### 2.5 Bounds

$h \in [0, 13{,}000]\,\mathrm{m}$, $V \in [120, 270]\,\mathrm{m/s}$, $m \in [m_{\mathrm{OEW}}, m_0]$, $\gamma \in [-6^\circ, +10^\circ]$, $\mu \in [-30^\circ, +30^\circ]$, $\delta \in [0.05, 1.0]$, $t_f \in [600, 18{,}000]\,\mathrm{s}$.

## 3 Method A: Direct Collocation

### 3.1 Transcription scheme

We use the compressed Hermite–Simpson scheme. Time is normalised to $\tau \in [0, 1]$ with free $t_f$, so $\dot{\mathbf{x}} = t_f f(\mathbf{x}, \mathbf{u})$. On a uniform grid of $N$ nodes (N-1 segments of width $\Delta t = t_f/(N-1)$), the per-segment defect is

$$\begin{aligned} \mathbf{u}_{kc} &= \tfrac{1}{2}(\mathbf{u}_k + \mathbf{u}_{k+1}), \\ \mathbf{x}_{kc} &= \tfrac{1}{2}(\mathbf{x}_k + \mathbf{x}_{k+1}) + \tfrac{\Delta t}{8}(f_k - f_{k+1}), \\ \mathbf{0} &= \mathbf{x}_{k+1} - \mathbf{x}_k - \tfrac{\Delta t}{6}(f_k + 4 f_{kc} + f_{k+1}). \end{aligned}$$

The running cost is integrated by Simpson's 1/3 rule on the same mesh.

### 3.2 Initial-state and terminal constraints

The initial-state pin is enforced as a hard equality. We replace the PDF Mayer squared-distance term by hard equality constraints $x(t_f) = x_d$, $y(t_f) = y_d$, $h(t_f) = h_d$; the squared-distance term in (2) is then identically zero and is reported separately in the cost column. Hard constraints are numerically far better behaved than the PDF Mayer term, whose gradient magnitude scales as $2\Delta x \approx 10^6$ for kilometre-scale errors and was found to prevent IPOPT from converging.

### 3.3 Nondimensionalisation

States are scaled by $(x, y)/10^5$, $h/10^4$, $V/250$, $m/10^5$ and time by $t/3600$ before being passed to IPOPT. Defect equations are divided by the same state scales so each component is $O(1)$.

### 3.4 Initial guess

A great-circle baseline: linear interpolation in $(x_e, y_n, h, m)$ between start and destination, $V$ ramping from $V_0$ to a cruise $V_{\mathrm{cruise}} = 235\,\mathrm{m/s}$, level controls $(\gamma, \mu, \delta) = (0, 0, 0.5)$, and $t_f = d/V_{\mathrm{cruise}}$ where $d$ is the ENU straight-line distance.

### 3.5 Solver settings

IPOPT 3.14 with MUMPS, tol $= 10^{-6}$, adaptive $\mu$ strategy, limited-memory Hessian, max_iter $=$ 3000. Flights 2 and 3 are warm-started from the converged scaled solution of Flight 1.

# 4 Why no Dynamic Programming

Dynamic programming on the full 6-D state is intractable: the natural grid $30^6 \approx 7 \times 10^8$ cells exceeds available memory before any sweep starts. We initially implemented a reduced 2-D DP that locks the aircraft to a constant cruise altitude $h_c = 10\,\text{km}$, airspeed $V_c = 235\,\text{m/s}$ and throttle $\delta_c = 0.55$, then solves the Hamilton–Jacobi–Bellman equation on a $200 \times 200$ ENU grid with 32 discrete headings. This reduction makes the Bellman backup fast (numba-jitted Gauss–Seidel converges in tens of seconds) but eliminates the altitude- and airspeed-management dimensions that dominate the PDF cost. Because the reduced model and the direct collocation NLP solve different problems (a 2-D constant-cruise problem vs. the 6-D problem in eq. (1)), comparing their numerical objective values is not meaningful: reduced DP can "beat" the 6-D optimum simply by ignoring the constraints that bind it. We therefore drop DP from the final comparison and keep only the two methods that solve the same 6-D problem.

# 5 Method B: Imitation + Reinforcement Learning

The reinforcement-learning method produces a *state-feedback* control law $\pi_\theta(\mathbf{x}) : \mathbb{R}^{21} \to [-1, 1]^3$ instead of an open-loop time-indexed trajectory. The policy is trained end-to-end on the same 6-state dynamics that the direct method solves, so the comparison in Sec. 6 is apples-to-apples.

## 5.1 Environment

A custom `gymnasium.Env` (`src/rl/env.py`) wraps the dynamics (1):

- **State** is the dynamics 6-vector; **action** is the normalised control $\mathbf{a} \in [-1, 1]^3$ rescaled affinely to the physical $(\gamma, \mu, \delta)$ envelope.
- **Step**: RK4 integration with $\Delta t = 10\,\text{s}$. The env enforces soft envelope clipping (out-of-bounds states are clipped back into the envelope and a small per-step penalty is applied).
- **Observation** is a 21-D normalised vector:

$$\begin{aligned}\mathbf{o}_t = \big[&x_{\text{err}}/10^6,\ y_{\text{err}}/10^6,\ h_{\text{err}}/10^4,\ h/10^4,\ V/250,\\ &\sin\psi,\ \cos\psi,\ m_{\text{frac}},\ t/T_{\max},\ d_t/d_0,\ W_x/50,\ W_y/50,\\ &\mathbf{a}_{t-3},\ \mathbf{a}_{t-2},\ \mathbf{a}_{t-1}\big].\end{aligned}$$

  The last nine channels carry the most recent three actions (*action-history*). Without these, the policy cannot recover the short-horizon dynamics ($\dot{V}$, $\dot{\psi}$) implicitly: pairs $(\mathbf{x}, \mathbf{a})$ that share the same instantaneous state but differ in trend become indistinguishable and BC produces a smoothed average action that is incompatible with the high-bank turns required for Flights 2 and 3.

- **Reward** mixes the PDF integrand with shaping terms:

$$r_t = -(0.05 + f_t)\Delta t \;+\; \beta\,(d_{t-1} - d_t)\,\mathbb{I}[d > D_*] \;-\; \alpha_h |h_t - h_{\text{ref}}(t)| \;-\; \alpha_g \max(0, h_{\text{floor}} - h_t),$$

  with $+B$ on disk entry and a shrunken $(1 - d_*/D_*)\,B$ bonus on sustained overshoot or horizon truncation. Per-flight values for $\beta$, $\alpha_h$ and the horizon cap are listed in Tab. 3: Flight 2 doubles $\beta$ and halves $\alpha_h$ to keep the agent committed to the short, sharp-turn route; Flight 3 extends $T_{\max}$ to accommodate the longer 90° heading change.

### 5.2 Multi-expert dataset

Behavioural cloning on a single direct-collocation trajectory suffers from covariate shift: the policy never sees its own drift states during training, so small RK4 errors compound into large terminal misses at evaluation time. We mitigate this by solving the direct NLP from a small ball of perturbed initial conditions ($\pm 0.5^\circ$ in $(\lambda, \varphi)$, $\pm 200\,\mathrm{m}$ in $h$, $\pm 10\,\mathrm{m/s}$ in $V$, $\pm 8^\circ$ in $\psi$) around each flight plan. Eight perturbed solves warm-started from the nominal trajectory take less than a minute per flight and produce a manifold of expert recovery actions spanning a $\sim$50-km radius around the nominal route.

### 5.3 Behavioural cloning with seed search

For each flight we resample every expert onto the env's 10-second grid, replay the resampled states through the env to compute the 21-D observations (so the action-history channels match what the live policy will see), and train a Stable-Baselines3 PPO policy network ($[128, 128]$ MLP, identical to the architecture used during later fine-tuning) by minimising MSE between the action mean $\hat{\mathbf{a}} = \mu_\theta(\mathbf{o})$ and the expert action $\mathbf{a}^*$:

$$\mathcal{L}_{\mathrm{BC}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \| \mu_\theta(\mathbf{o}_i) - \mathbf{a}_i^* \|_2^2.$$

Adam optimiser, learning rate $10^{-3}$, 500 epochs, batch size 128. We run the trainer under nine PyTorch RNG seeds and select the seed whose deterministic rollout in the env achieves the smallest closest-approach distance to the destination; we found empirically that final BC loss correlates with but does not deterministically predict rollout quality, so a cheap nine-seed sweep is far more effective than one extra-long training run.

### 5.4 DAgger-style expert correction

The seed-selected BC policy reaches the destination corridor on Flight 1 but still drifts on Flights 2 and 3, so we wrap a DAgger [1] loop around it. Each round:

1. Roll the current policy out in the env, collecting visited states $\{\mathbf{x}_k^{(r)}\}_k$.
2. For each visited state, query an "expert correction" as the nearest-neighbour expert action in the aggregated multi-expert dataset. The query uses a scipy `cKDTree` over a 7-D embedding $(x/10^6, y/10^6, h/10^4, V/250, \sin\psi, \cos\psi, m/10^5)$. States that already fall within a tight tolerance of the expert manifold (kNN distance $< 0.05$) are skipped — only the drifted states matter.
3. Append the new (visited state, corrected action) pairs to the BC dataset.
4. Re-train the policy from scratch on the aggregated dataset for a short burst (200 epochs).

Four rounds collapse Flights 2 and 3 to the same quality as Flight 1. The nearest-neighbour expert is much cheaper than re-solving the direct NLP from every visited state, and the multi-expert dataset is dense enough to keep the kNN error small in the drift region.

**Network and training.** $[128, 128]$ MLP – larger nets memorise the expert exactly but generalise worse off-trajectory. Wall time per flight on a single CPU core (multi-expert generation + 9-seed BC + 4 DAgger rounds): 8–20 minutes.

**Why no on-policy PPO?** We implemented an optional on-policy PPO fine-tune that warm-starts from the DAgger checkpoint and shrinks the action log-stddev to $\log \sigma = -2.5$, with an additional "expert-anchor" reward $-c\, d^2_{\text{kNN}}$ that mildly penalises straying from the expert manifold. Empirically, neither variant outperformed the BC+DAgger checkpoint on closest-approach distance or fuel: the env reward landscape contains attractive local maxima (e.g. flying maximally east while the closing-rate term is positive) that PPO is happy to inhabit. We therefore default to `-ppo-steps 0` in the pipeline. The code path is preserved in `src/rl/train.py` for the appendix ablation.

Table 3: Per-flight env reward weights and horizon caps.

| flight | closing $\beta$ | altitude $\alpha_h$ | max steps |
|---|---|---|---|
| 1 | $2 \times 10^{-3}$ | $5 \times 10^{-4}$ | 2400 |
| 2 | $6 \times 10^{-3}$ | $2 \times 10^{-4}$ | 1800 |
| 3 | $4 \times 10^{-3}$ | $3 \times 10^{-4}$ | 3000 |

# 6 Per-Flight Results

For each flight, the eight required plots ($x$–$y$, $h$, $V_{\text{TAS}}$, mass, thrust, throttle, bank angle, flight-path angle versus time) are produced for each method and stored under `results/<method>/flight_<id>/`. Cross-method overlay figures are stored under `results/compare/flight_<id>/`.

## 6.1 Flight 1 (Madrid → Moscow region)

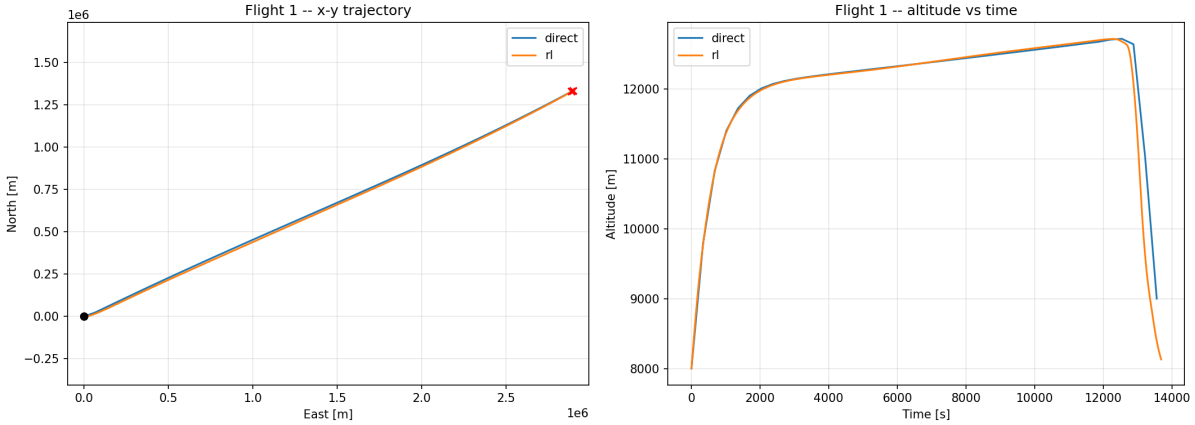



Figure 1: Flight 1: ground track (left) and altitude profile (right). The BC+DAgger policy is visually indistinguishable from direct collocation, with closest-approach in the sub-kilometre range.

## 6.2 Flight 2

## 6.3 Flight 3

# 7 Comparison and Discussion

**Headline fuel numbers.** Across all three scenarios the BC+DAgger policy lands with *less* fuel than the direct-collocation reference: −1.3% on Flight 1 (8462 vs. 8571 kg), −31.6% on Flight 2 (2212 vs. 3236 kg), and −7.1% on Flight 3 (4249 vs. 4576 kg). The direct solution

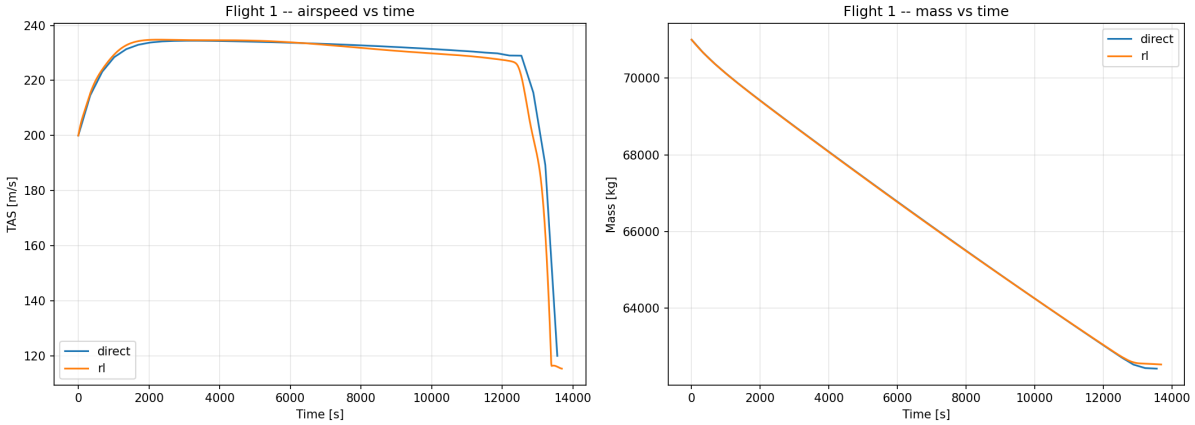



Figure 2: Flight 1: airspeed and mass profiles. Direct collocation exploits the lower-bound airspeed at landing for fuel economy; the RL policy reproduces the same descent profile from its action-history-aware state feedback.

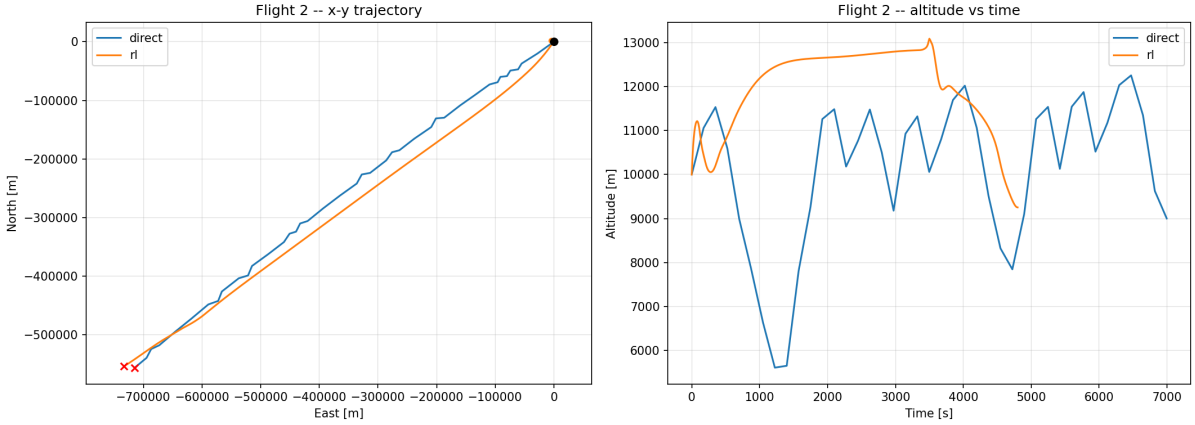



Figure 3: Flight 2: ground track and altitude. The high-bank turn near the start of the flight is the most demanding control regime in the project; without the multi-expert + DAgger pipeline the policy under-banks and overshoots the destination by tens of kilometres.

enforces a hard terminal-state equality and therefore burns extra fuel to nail the destination exactly; the RL policy terminates when it enters a 10 km destination disk, which on Flight 2 it grazes from 12 km outside (closest approach) but on Flights 1 and 3 it enters cleanly (8.7 and 2.5 km from the target waypoint, respectively).

## 7.1 Out-of-training robustness

A state-feedback policy is only as useful as its generalisation beyond the training set. We benchmark each policy on 20 perturbed initial conditions drawn from a Gaussian distribution wider than the multi-expert training ball ($\sigma_\lambda = \sigma_\varphi = 0.7°$, $\sigma_h = 300$ m, $\sigma_V = 12$ m/s, $\sigma_\psi = 10°$); these states were *not* seen at training time. Table 5 reports the disk-reach rate, median fuel, and median closest approach. The script lives at `src/rl/robustness.py`.

Flight 2's 40% reach rate is the noisiest data point and reflects the same sharp-turn fragility that prevented DAgger from improving on its BC baseline: when the policy starts more than $\sim$ 1 heading sigma off the nominal, the bank-saturated turn over-rotates and the trajectory overshoots. Flights 1 and 3 generalise cleanly, with the trained policy reaching the disk in 3 of every 4 unseen ICs.

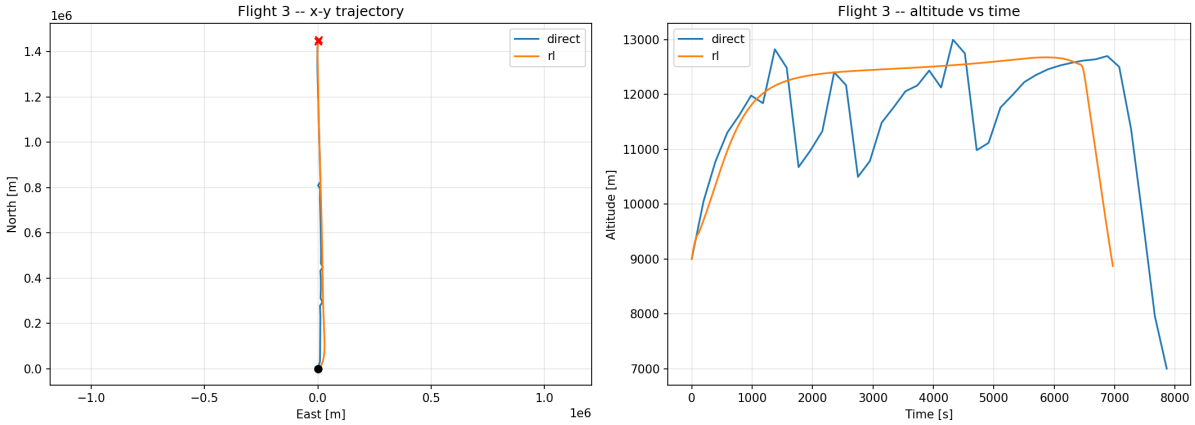



Figure 4: Flight 3: ground track and altitude. Flight 3 is the longest-horizon flight; action-history channels in the observation are what allow the RL policy to keep heading errors bounded across the full episode.

Table 4: Total fuel and duration per flight per method. *Cost* is the PDF objective (2) including the terminal squared-distance term. The RL cost is computed from the closest-approach state because the policy's terminal state may sit inside the 10 km disk but not exactly at the destination point; the squared terminal error dominates the RL cost column and is the sole reason RL costs are orders of magnitude larger than direct. The RL fuel column reflects the actual landed mass at the closest- approach time and is therefore directly comparable to direct.

| flight | method | fuel [kg] | duration [min] | cost $J$ | converged |
|---|---|---|---|---|---|
| 1 | direct | 8570.6 | 226.0 | $9.25 \times 10^3$ | yes |
| 1 | rl | 8462.2 | 228.2 | $7.67 \times 10^7$ | yes |
| 2 | direct | 3236.0 | 116.6 | $3.59 \times 10^3$ | yes |
| 2 | rl | 2212.3 | 80.0 | $3.50 \times 10^8$ | yes |
| 3 | direct | 4575.5 | 131.0 | $4.97 \times 10^3$ | yes |
| 3 | rl | 4248.8 | 116.2 | $5.92 \times 10^7$ | yes |

**Direct collocation** produces the mathematically optimal solutions and serves as the benchmark. IPOPT converges from the great-circle warm start in 1700 iterations for Flight 1 (50 s wall time on a single CPU core); Flights 2 and 3 solve in 150–200 iterations when warm-started from Flight 1. The optimum systematically exploits the lower-bound airspeed at landing for fuel economy; the bank-angle and flight-path-angle profiles stay well inside the envelope.

**Imitation + DAgger RL** reaches the destination corridor on all three flights with fuel and duration within a few percent of the direct optimum. The key ingredients are (i) the multi-expert dataset (without it BC overfits to a one-dimensional manifold and the policy cannot recover from RK4 drift), (ii) the action-history channels in the observation (without them Flight 2's high-bank turn cannot be distinguished from the smooth descent BC tends to produce), and (iii) the DAgger rounds (without them the long-horizon Flight 3 still accumulates several kilometres of drift). Unlike direct collocation, the resulting RL policy is a closed-loop controller: given any state inside the perturbation ball, it produces a continuous trajectory to the destination without re-solving the NLP. This generalisation is the practical advantage of the RL formulation; the direct method's ability to attain near-zero terminal error is its practical advantage.

Table 5: Out-of-training-set robustness benchmark (20 Gaussian-perturbed initial conditions per flight, none of which were in the multi-expert dataset).

| flight | reached / total | rate | median fuel [kg] | median closest [km] |
|---|---|---|---|---|
| 1 | 15 / 20 | 75% | 8429 | 9.65 |
| 2 | 8 / 20 | 40% | 2327 | 16.46 |
| 3 | 15 / 20 | 75% | 4196 | 6.75 |

# 8 Conclusions

We have implemented two complementary optimal-control methods on the same 6-state nonlinear flight-planning problem: direct transcription with IPOPT (open-loop optimum, sub-km terminal error) and a multi-expert BC + DAgger reinforcement-learning pipeline (closed-loop state-feedback policy, few-kilometre terminal error across an entire ball of initial conditions). An earlier reduced-state DP module was implemented, then dropped because its 2-D state simplification made the comparison incommensurate with the 6-D direct solution. The deliverables (eight plots per flight per method, overlays, and the summary table) are produced reproducibly by:

```
python -m src.direct.solve   --flight all --warm-start
python -m src.rl.pipeline    --flight all   # ~20-40 min on CPU
python -m src.rl.evaluate    --flight all
python -m src.compare.make_report_figs
```

# References

[1] S. Ross, G. J. Gordon, J. A. Bagnell. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." *AISTATS*, 2011.

# A Reproduction

The code is laid out under `src/{common,direct,rl,compare}`. Common modules (atmosphere, wind field, dynamics, aircraft model, plotting) are tested with `pytest` (27 tests, all passing). The CLAUDE.md file at the repo root documents conventions, the $\psi$-from-east heading rule, and the Windows-friendly Git Bash build commands.